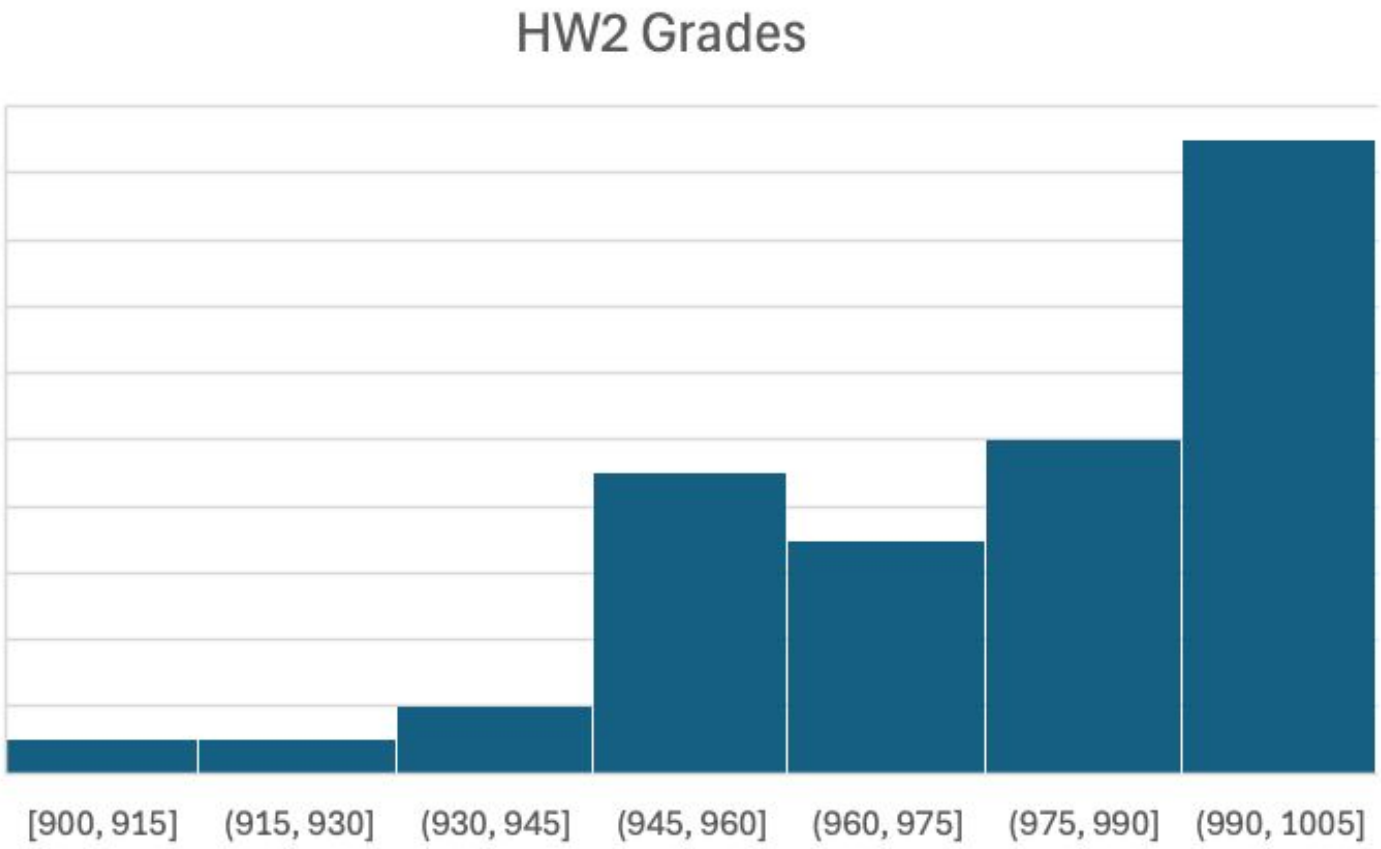
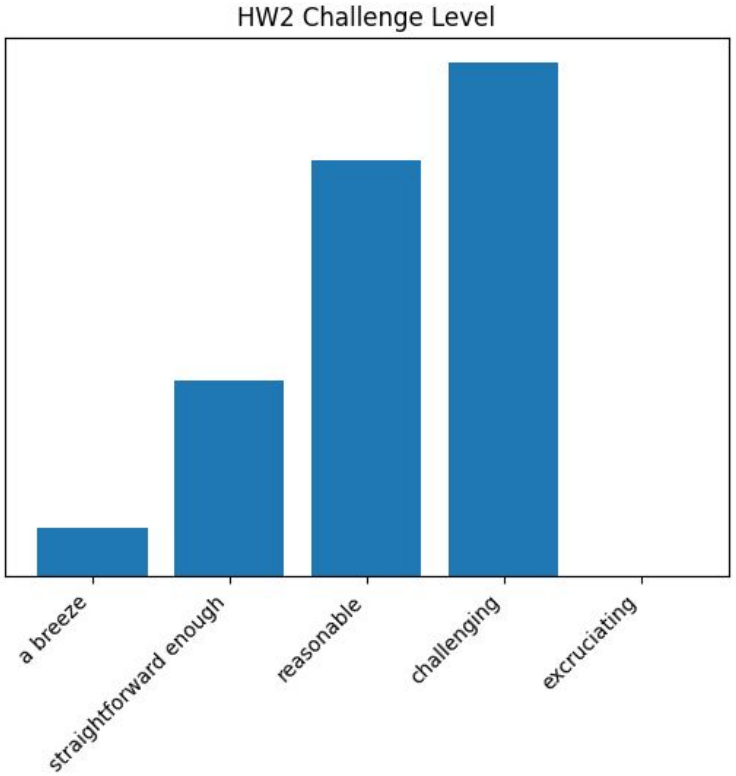
CSE 493L Section 4

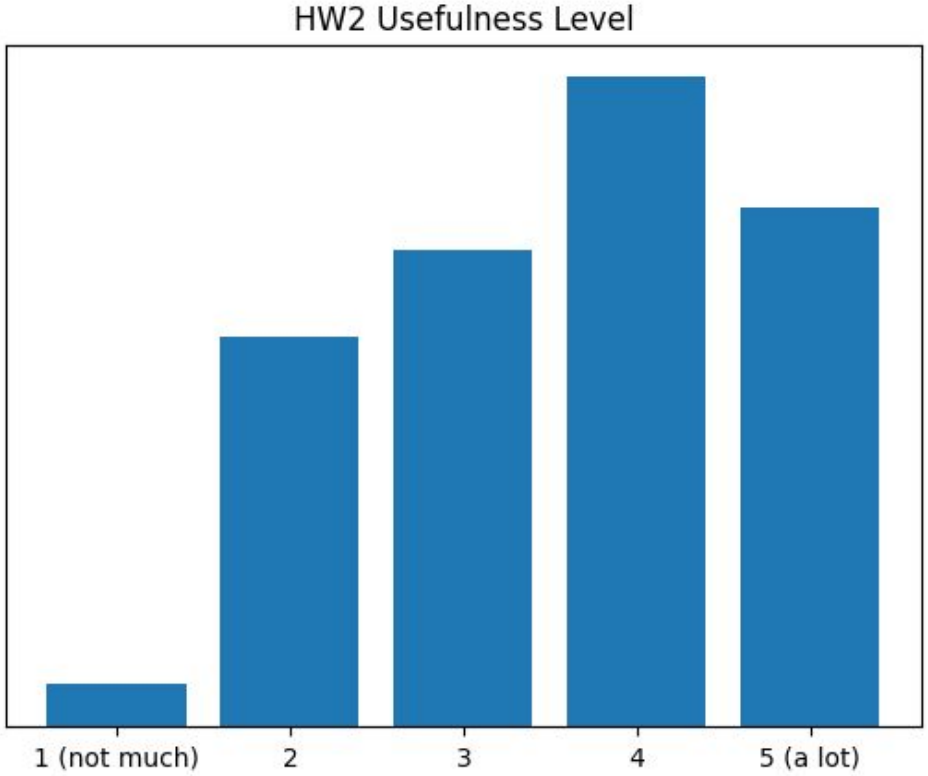
A Bunch of Rust Intuition

Looking Back on Homework 2



Looking Back on Homework 2





Homework 4: Due Dates

- **Wednesday, February 4th**: code, questions - 2 day grace period

Consists of multiple short exercises, to get you comfortable with Rust's:

- Compiler errors
- Syntax
- Common types
- …and much more!

Homework 4: Required Reading

- README for this homework is quite long… take it one exercise at a time.

README.md

Homework 4: Introduction to Rust

We have a list of exercises with some commented-out starter code. Your job is to uncomment the relevant sections, implement the described functionality, and pass the test cases provided for each file.

The goal is to get hands-on experience writing Rust code for the first time, and to run into common compiler errors that do not appear in other languages. A description of each exercise follows, alongside a few tips for implementing each one. View `Working-With-Rust.md` for useful installation instructions.

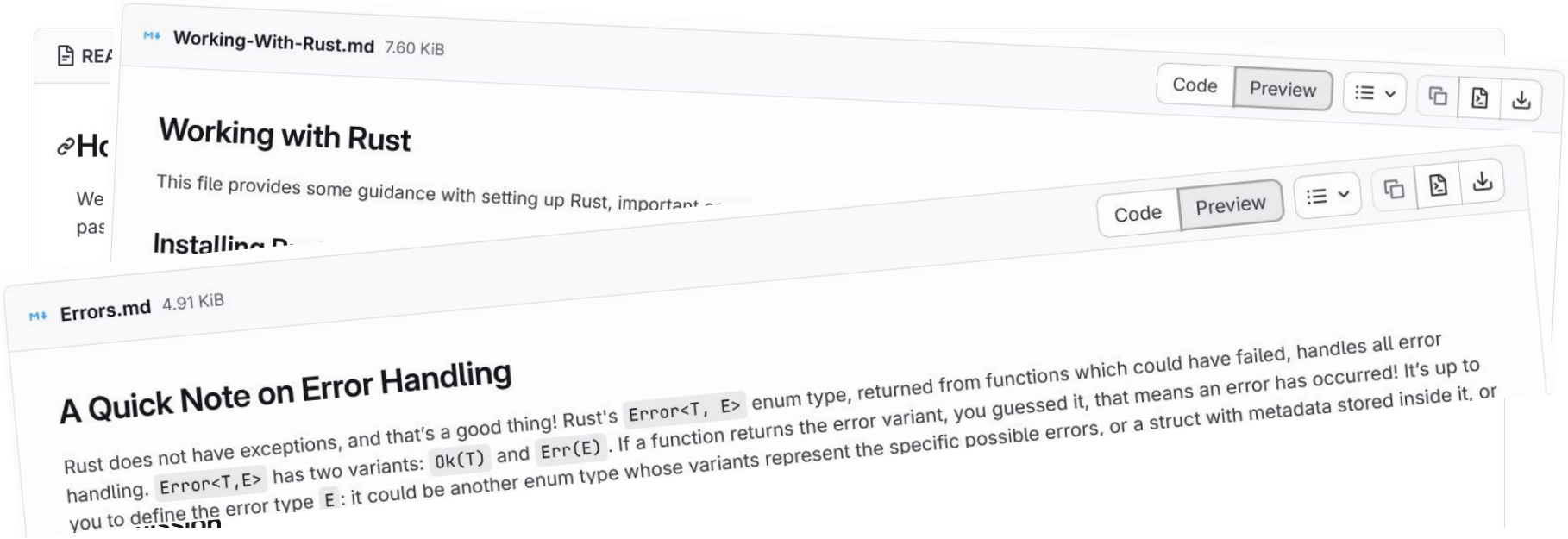
The goal of this homework is to:

- Provide you with experience working with Rust's syntax.
- Debug issues brought up by the borrow checker.
- Utilize Rust's type system.

Submission

Homework 4: Required Reading

- README for this homework is quite long… take it one exercise at a time.
  - Be aware of Working-With-Rust.md and Errors.md for even more high-level information



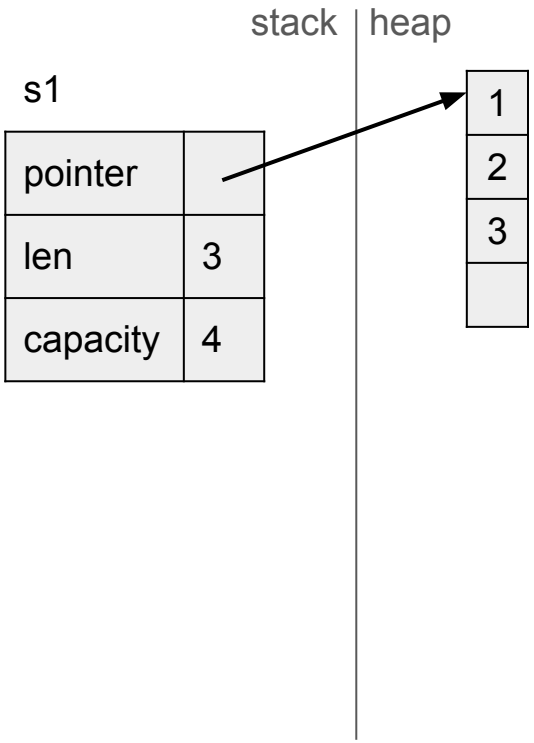
Check-in

- Before we go on to talk about more Rust stuff: *any questions?*

Memory

```
let s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: Vec<u32> = s1;
```

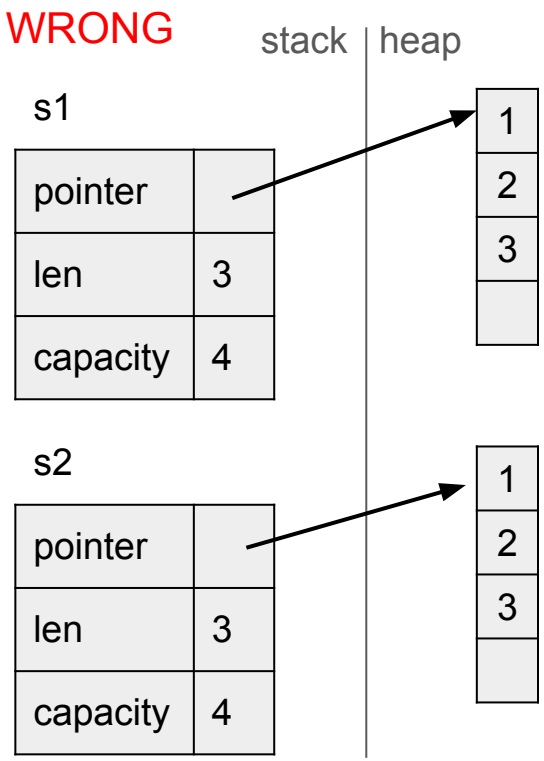
- Like in C, all variables are on the stack.
- They may contain pointers to the heap.
- What happens when we do **`s2 = s1`**?



Wrong Guess: Deep Clone

```
let s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: Vec<u32> = s1;
```

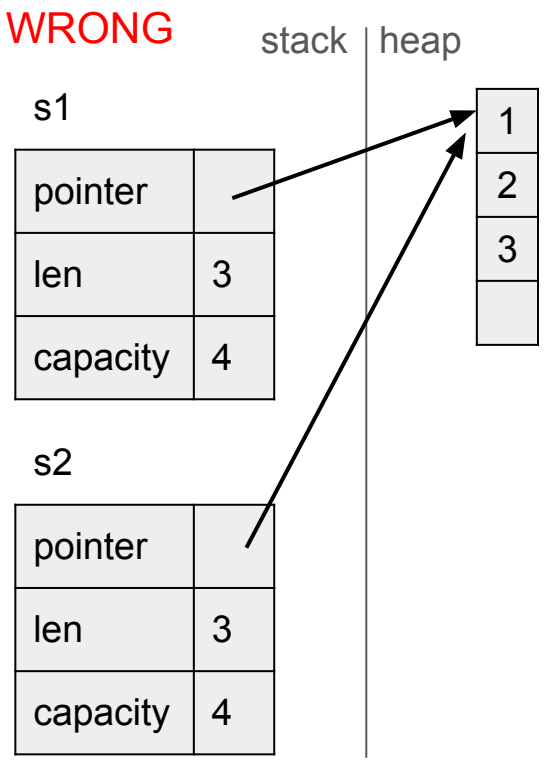
- In reality, = always does a bitwise memcpy of only the stack variable.
- (well, under the hood, the compiler might optimize that away.)



Wrong Guess: Shallow Copy

```rust
let s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: Vec<u32> = s1;
```

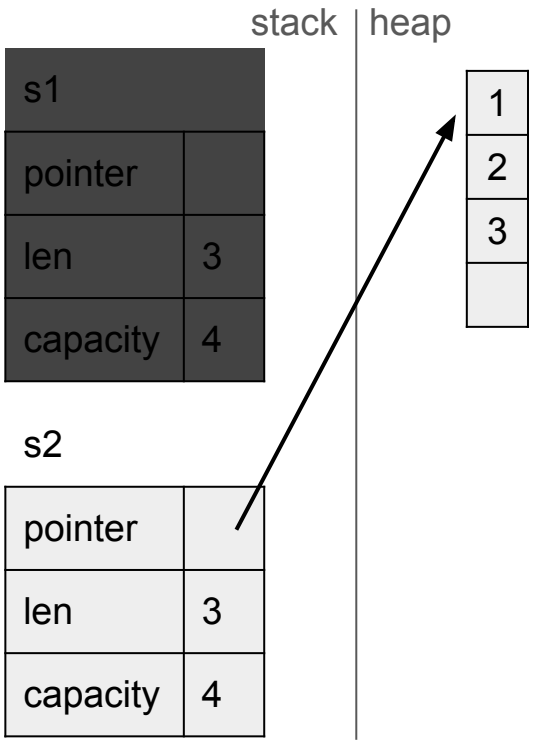
- Won't happen because **Vec** doesn't implement the **Copy** trait.
- If this happened, when `s1` and `s2` go out of scope, their **drop()** methods will be called.
- Both would free the same heap block.
- Double-free bug.



Actual Behavior

```rust
let s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: Vec<u32> = s1;
```

- The compiler ensures **`s1`** cannot be used again.
- **`s2`** becomes the new *owner*, so it will free the heap block when it goes out of scope.
- Called a *move* from **s1** into **s2**.



What Happens On = ?

Move

- If type doesn’t implement **Copy** trait.
- Applies to **Vec**, **String**, etc.
- Bitwise memcpy.
- Compiler invalidates old variable.
- Transfers *ownership*.

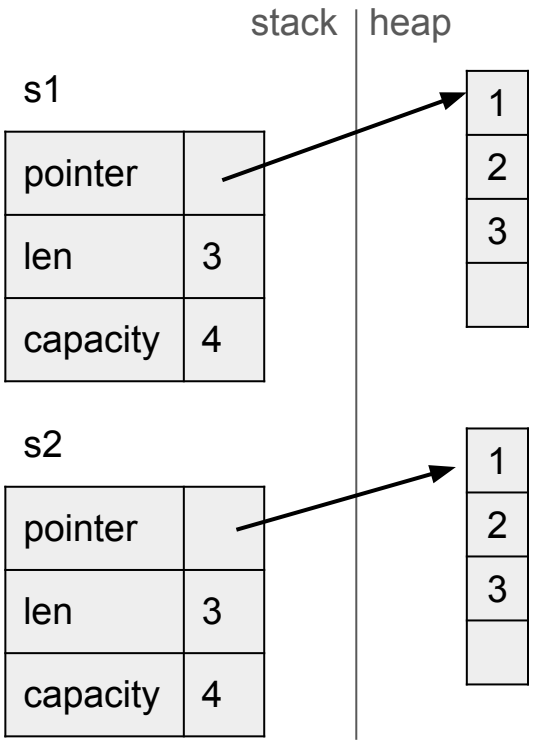
Copy

- If type implements the **Copy** trait.
- Applies to primitive data.
- Also bitwise memcpy.
- Old variable still usable.

Clone

```
let s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: Vec<u32> = s1.clone();
```

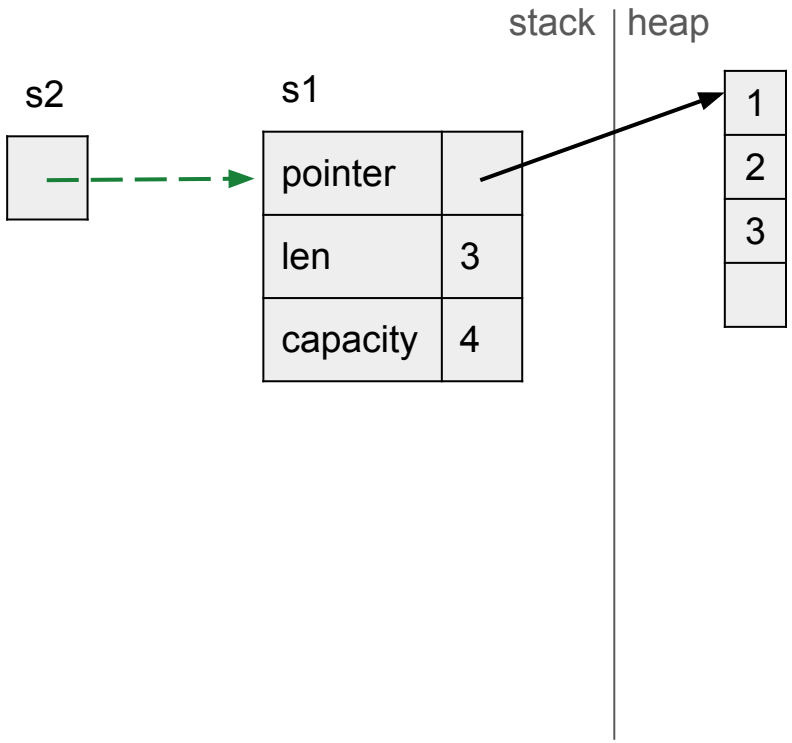
- Types can define a custom **.clone()** method.
- For **Vec**, does a deep copy.
- Now **s1** and **s2** are both *owners* of different heap blocks.



References

```rust
let mut s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: &mut Vec<u32> = &mut s1;
```

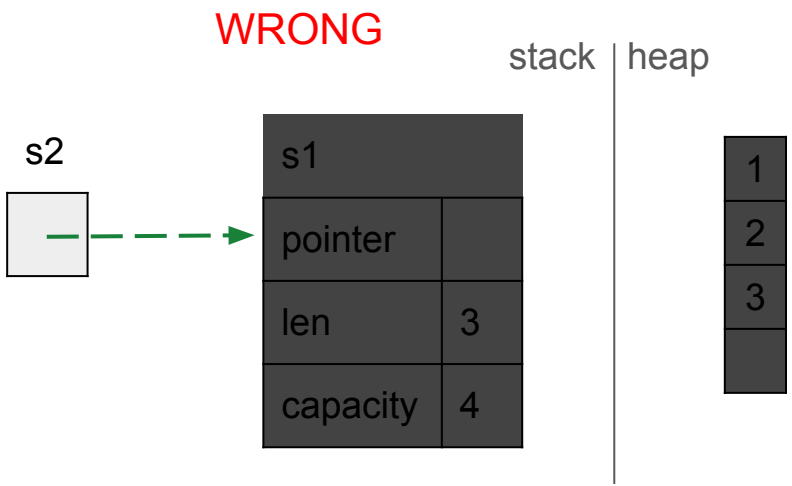
- Here, **s2** *borrows* **s1**.
- Nothing will happen when **s2** is dropped.
- **s1** is the *owner*, so when it's dropped, it will free the heap block.
- Rule: either many immutable references OR 1 mutable reference.



Lifetimes

```rust
let mut s1: Vec<u32> = Vec::from([1, 2, 3]);
let s2: &mut Vec<u32> = &mut s1;
drop(s1);
dbg!(s2); // compile error!
```

- A reference cannot outlive the lifetime of the variable it's pointing to.
- Lifetime is duration from creation to last use.
- Rust prevents dangling references.



References: Static and Dynamic Semantics

Why does the left program compile, but not the right program?

```rust
let mut s = MyStruct {
    a: 10,
    b: 20
};

let a_ref = &mut s.a;
let b_ref = &mut s.b;

println!("{}, {}", a_ref, b_ref);
```

```rust
let mut v = Vec::from([10, 20]);

let a_ref = &mut v[i];
let b_ref = &mut v[j];

println!("{}, {}", a_ref, b_ref);
```

References: Static and Dynamic Semantics

Why does the left program compile, but not the right program?

```rust
let mut s = MyStruct {
    a: 10,
    b: 20
};

let a_ref = &mut s.a;
let b_ref = &mut s.b;

println!("{}, {}", a_ref, b_ref);
```

```rust
let mut v = Vec::from([10, 20]);

let a_ref = &mut v[i];
let b_ref = &mut v[j];

println!("{}, {}", a_ref, b_ref);
```

`v[i]` and `v[j]` could be the same element if `i == j`!

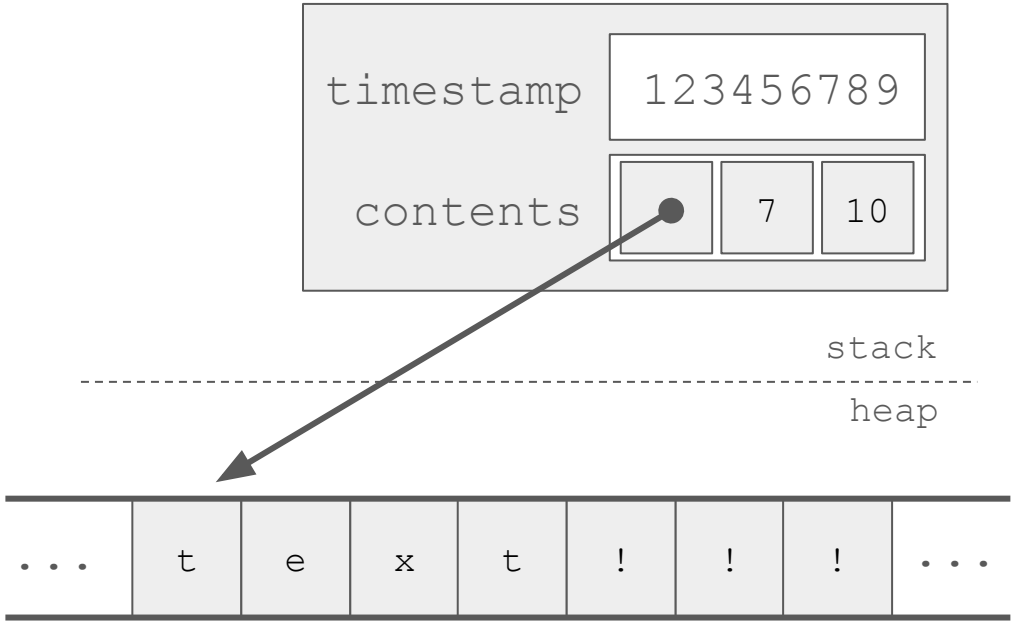
Structs Demo

- Declaring struct.
- Functions on structs.
- Methods.
- Self argument.

Ownership and Consumption

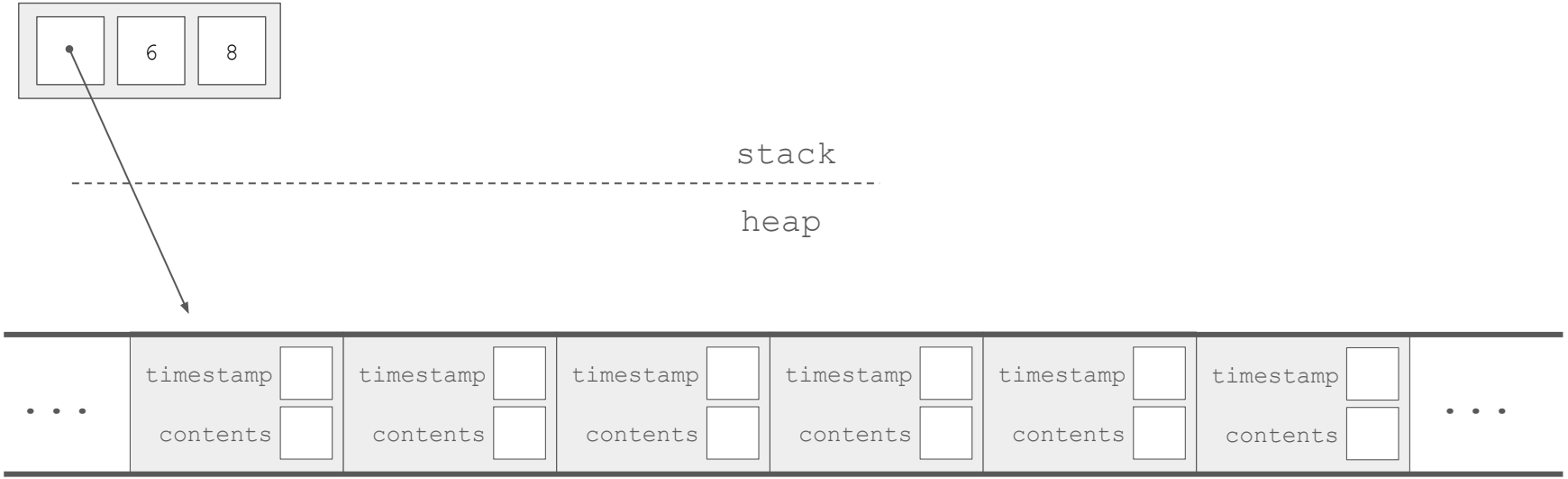
Let's define a new non-`Copy`, `Comment` struct.

```
struct Comment {
    timestamp: u64,
    contents: String,
}
```



Ownership and Consumption

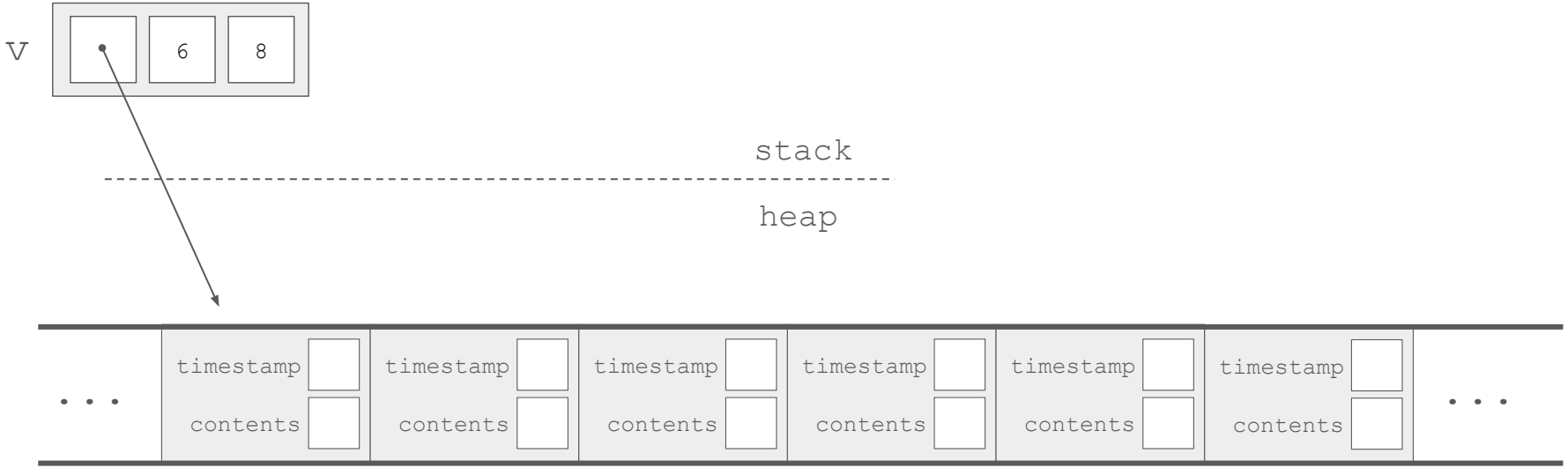
Let's create a `Vec<Comments>`



Ownership and Consumption

Let's create a `Vec<Comments>`

… and assign it to a variable `v`

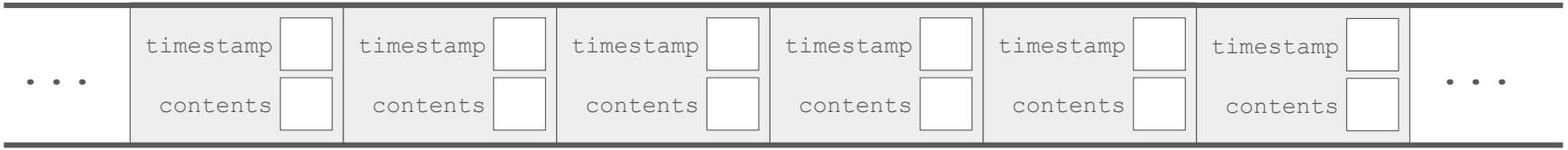


Ownership and Consumption

Let's create a `Vec<Comments>`

… and assign it to a variable `v`

```
let v: Vec<Comment> = Vec::from([
    Comment {
        timestamp: 123,
        contents: "some text".into()
    },
    Comment {
        timestamp: 456,
        contents: "different text".into()
    },
    Comment {
        timestamp: 789,
        contents: "something else".into()
    },
        ...
]);
```

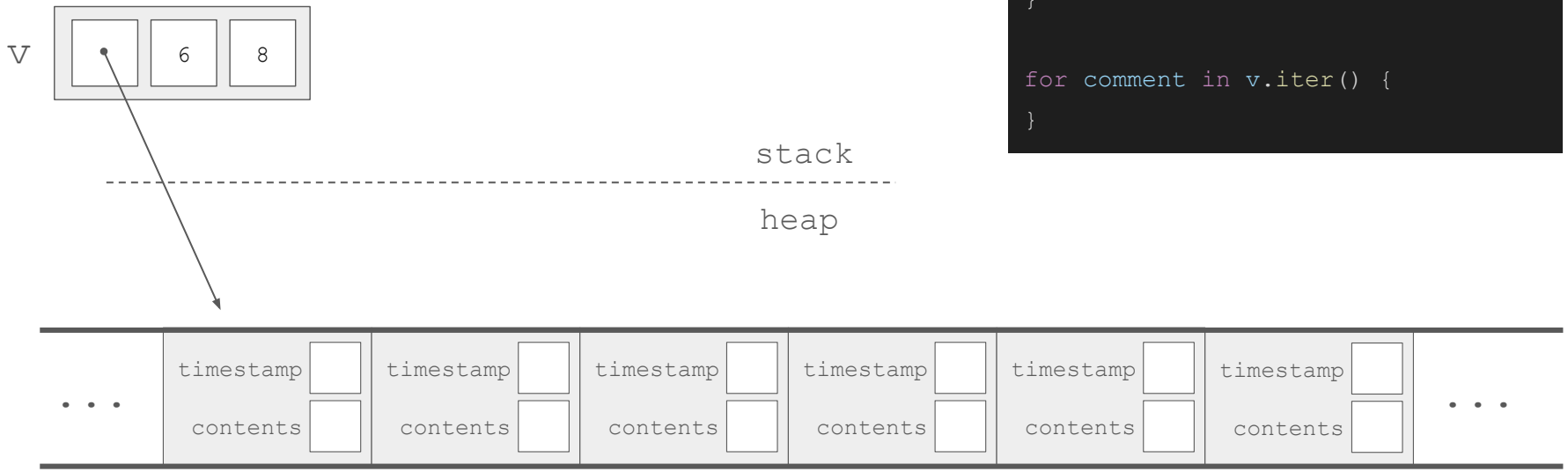


Ownership and Consumption

Let's think about how we can use an iterator over this vector

Note: these two are equivalent! Rust implicitly calls `.iter()` in this case.

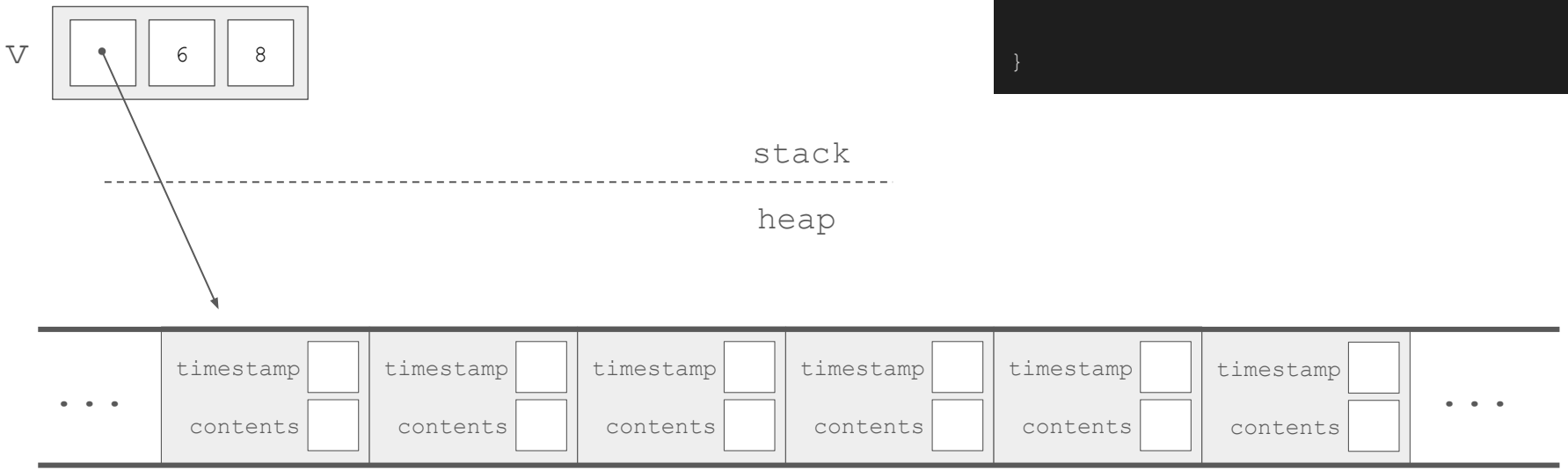
```
for comment in &v {
}

for comment in v.iter() {
}
```



Ownership and Consumption

Let’s think about how we can use an iterator over this vector

```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

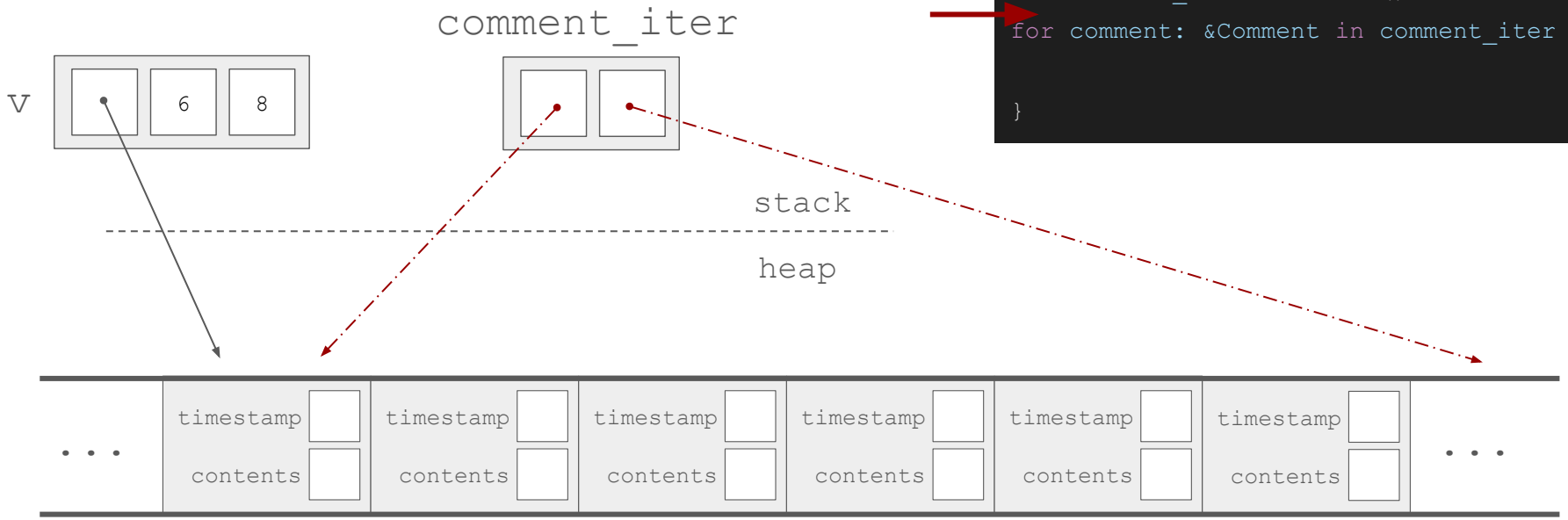


Ownership and Consumption

Note: the dot-dash red arrows are actual pointers, not references, you don't need to worry about the details…

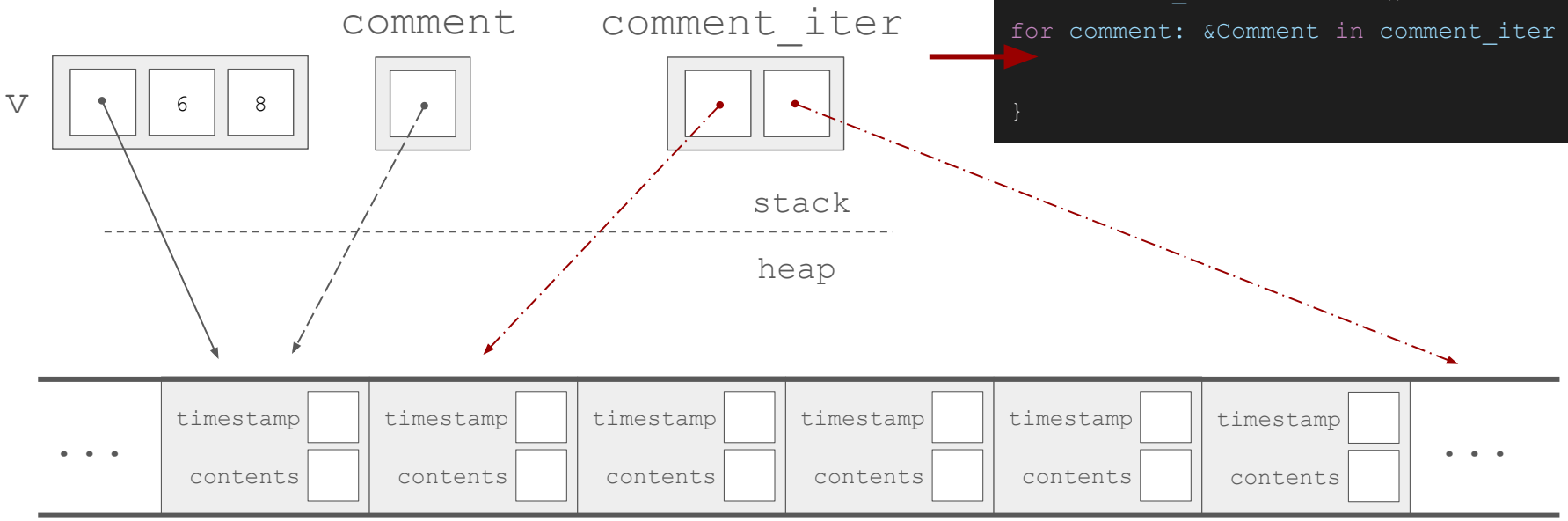
Let's think about how we can use an iterator over this vector

```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```



Ownership and Consumption

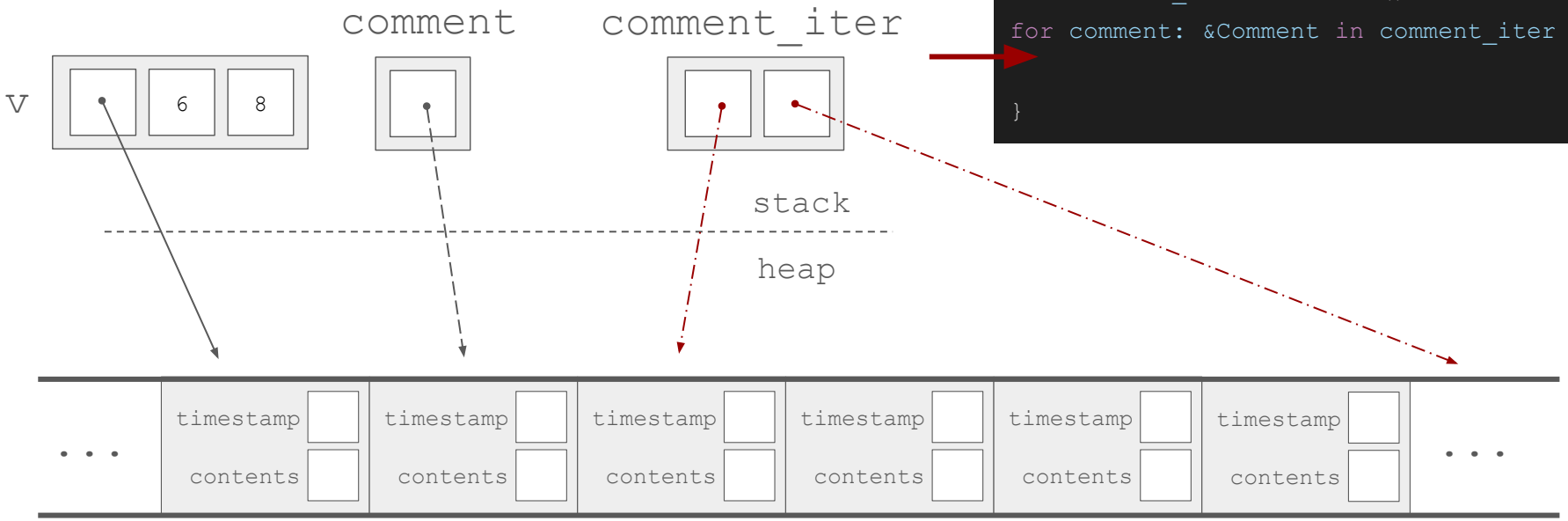
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

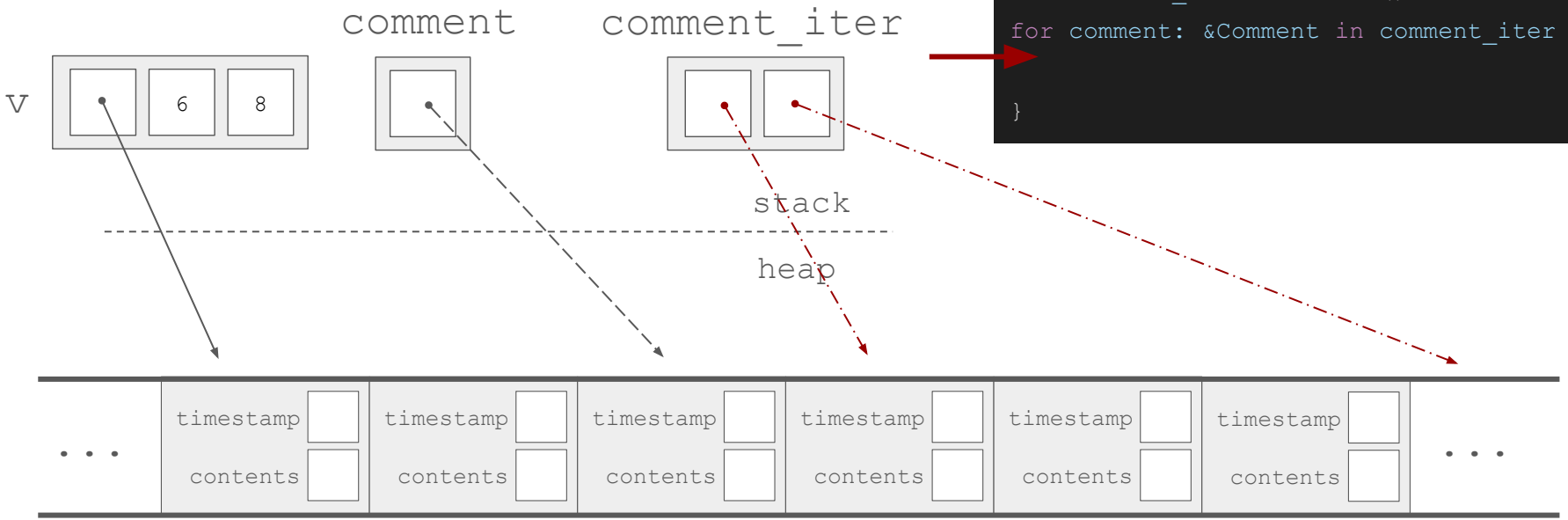
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

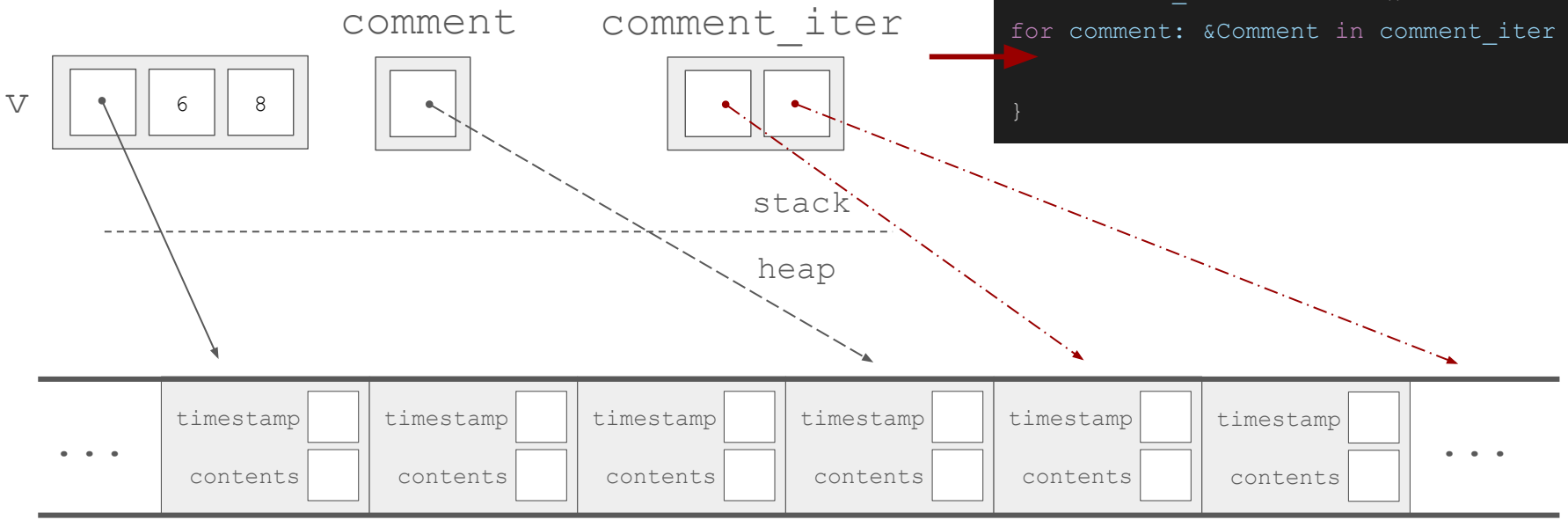
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

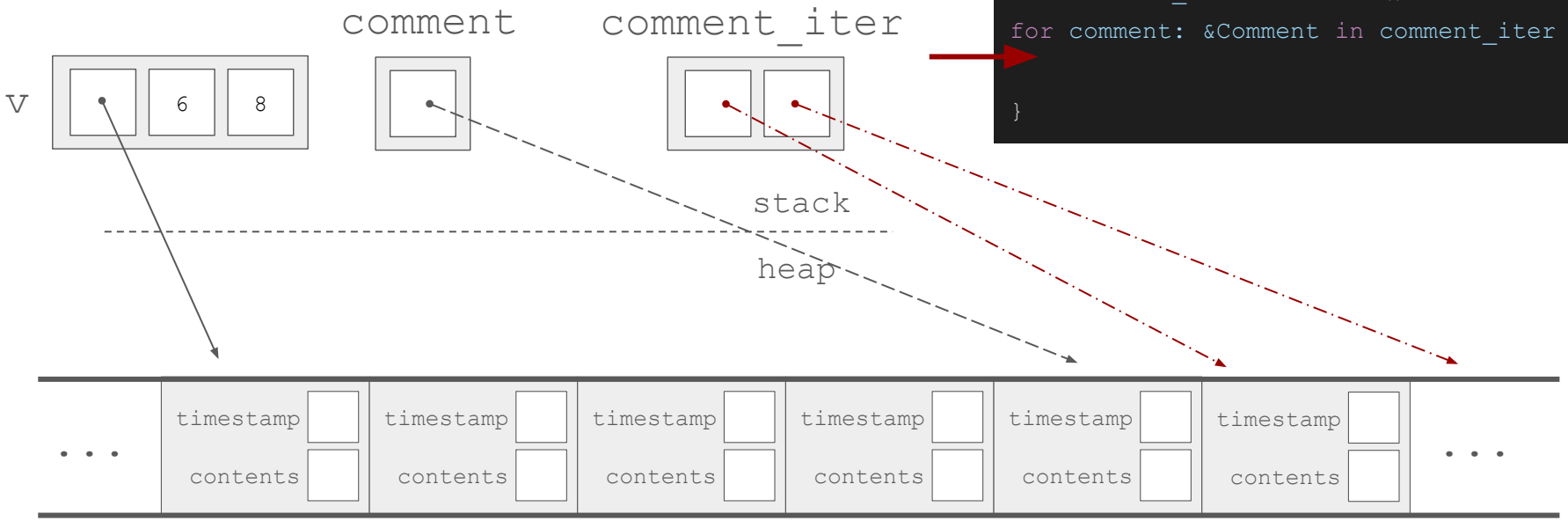
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

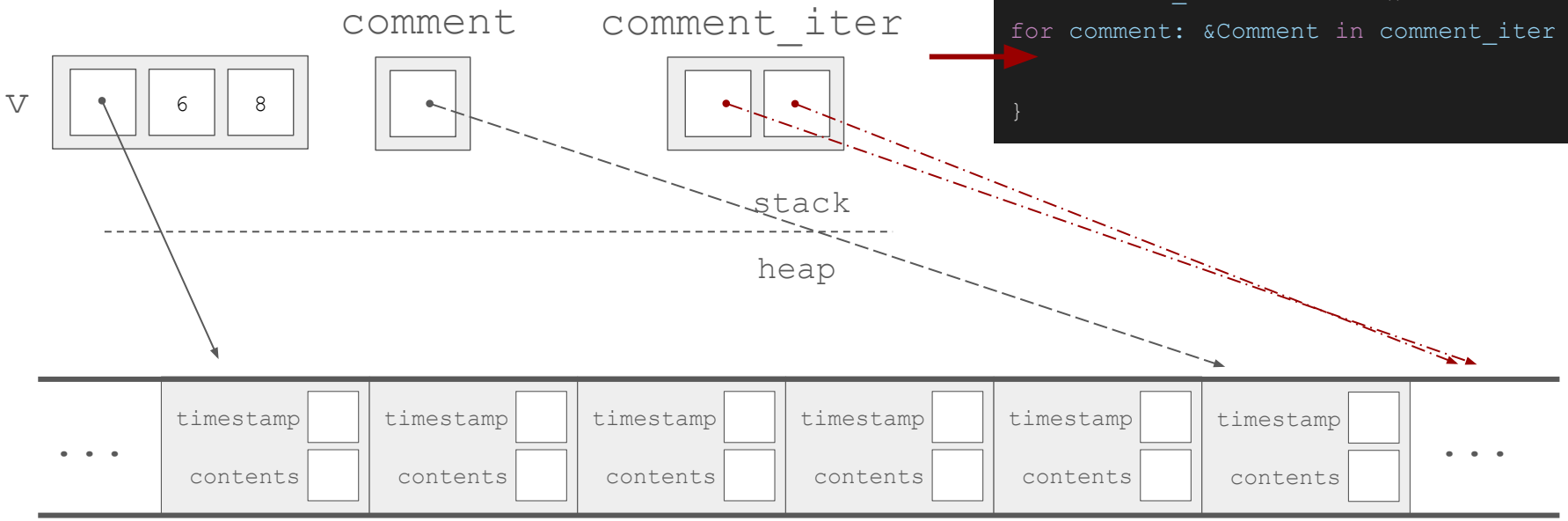
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

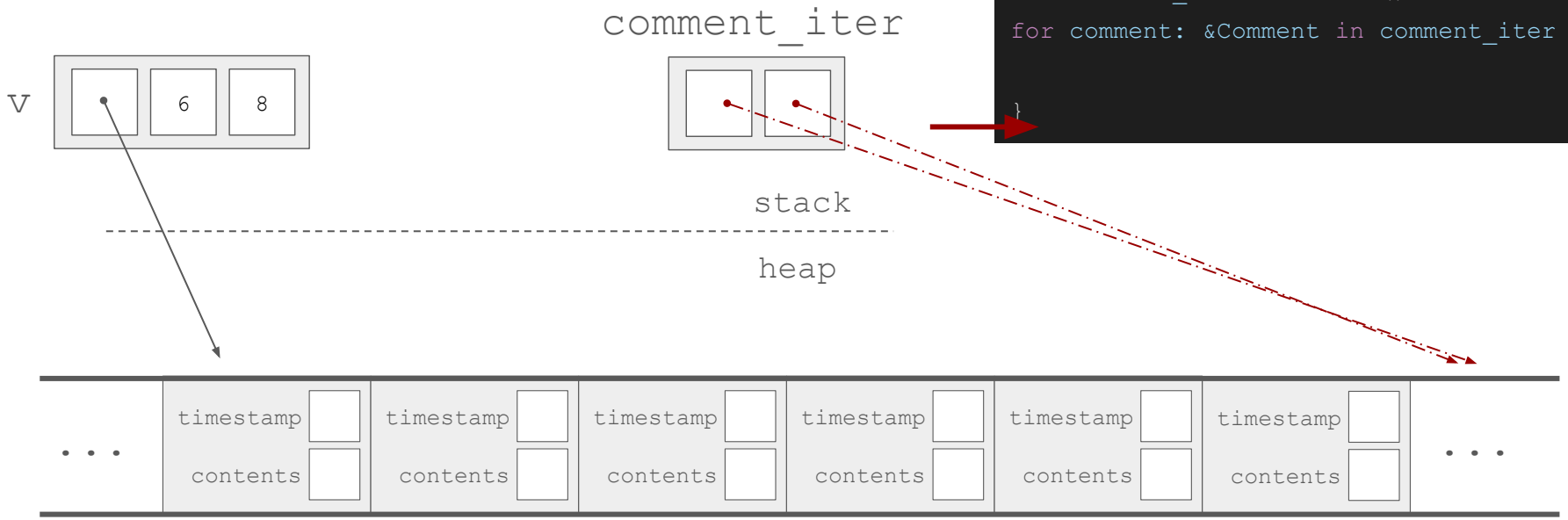
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

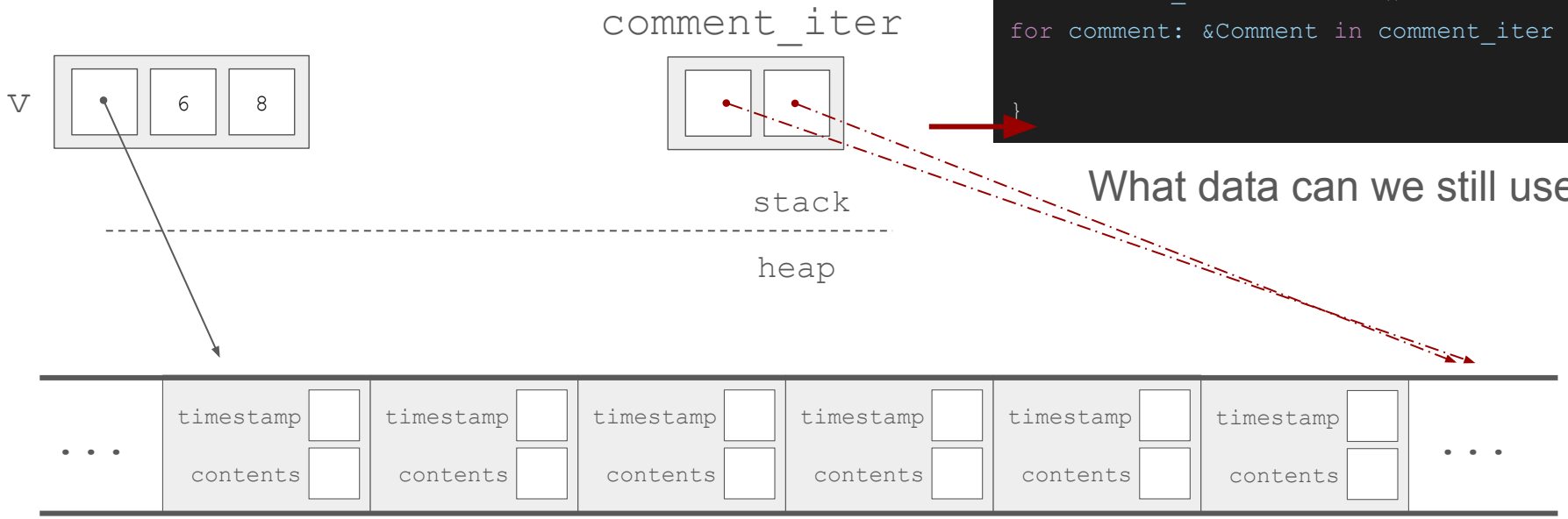
Let’s think about how we can use an iterator over this vector




```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

Ownership and Consumption

Let’s think about how we can use an iterator over this vector



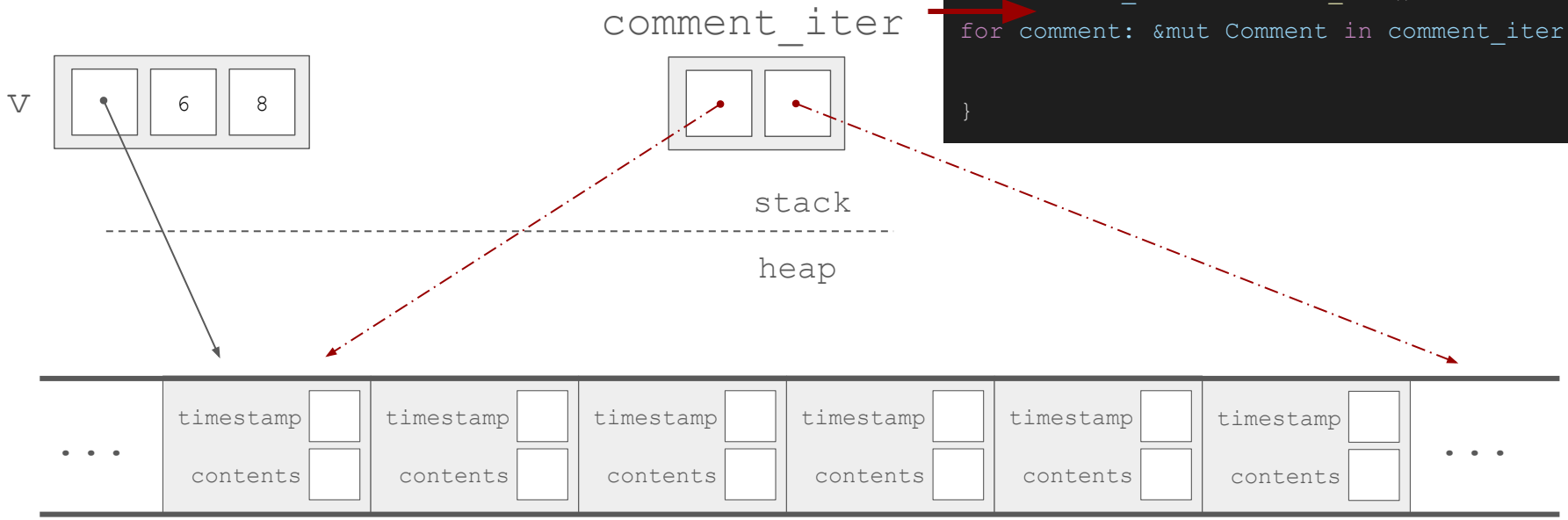

```
let comment_iter = v.iter();
for comment: &Comment in comment_iter {

}
```

What data can we still use?

Ownership and Consumption

Let's think about how we can use a *mutable* iterator over this vector

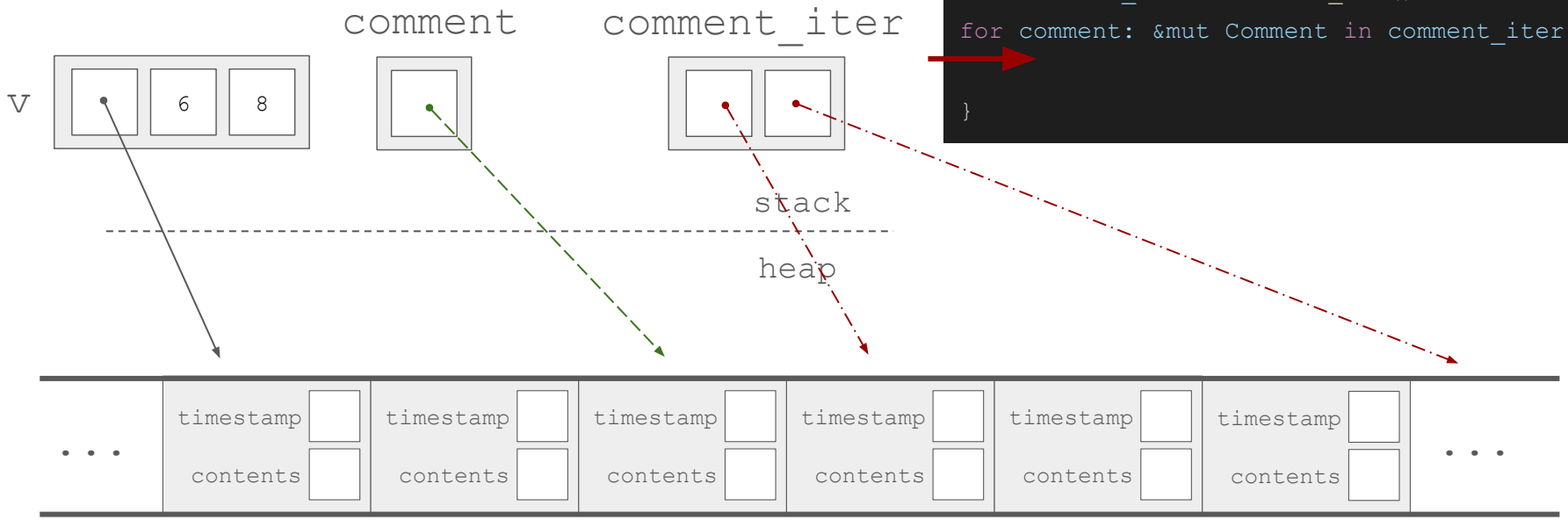
```
let comment_iter = v.iter_mut();
for comment: &mut Comment in comment_iter {

}
```



Ownership and Consumption

What restrictions are in place inside this for loop?

Let's think about how we can use a *mutable* iterator over this vector

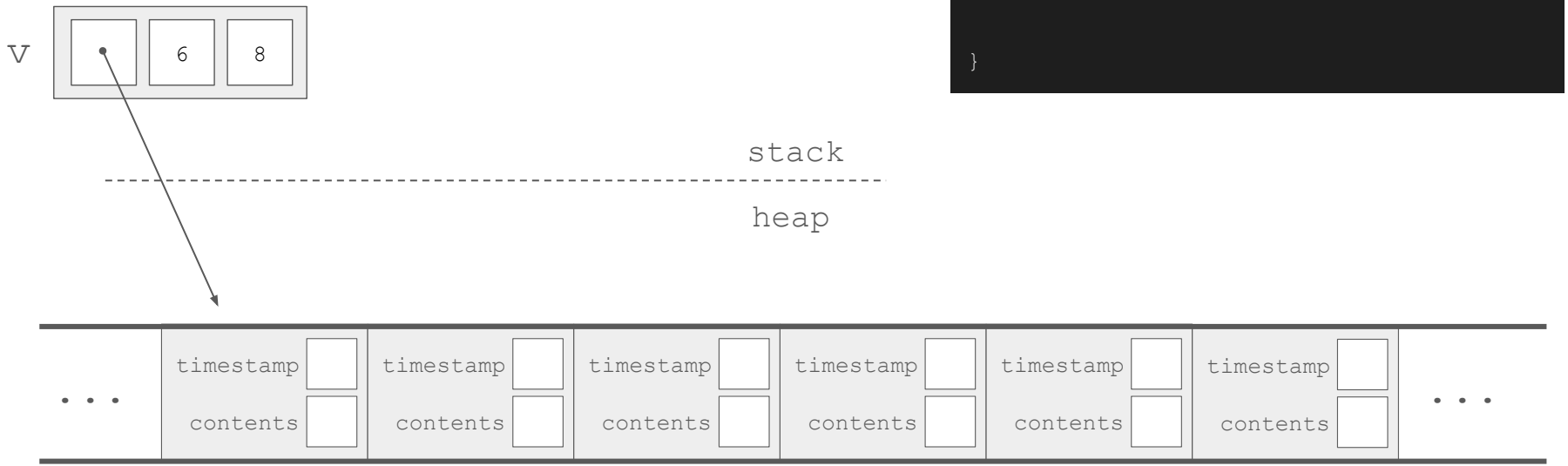



```
let comment_iter = v.iter_mut();
for comment: &mut Comment in comment_iter {

}
```

Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector

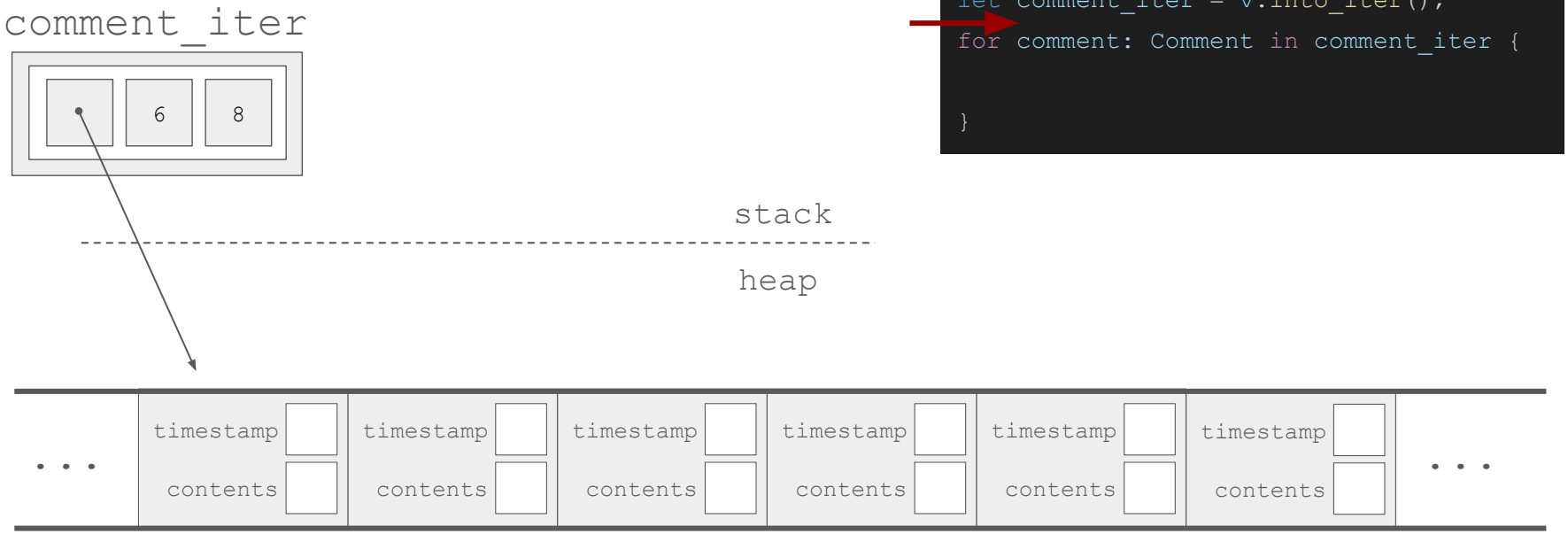
```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```



Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector

```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```

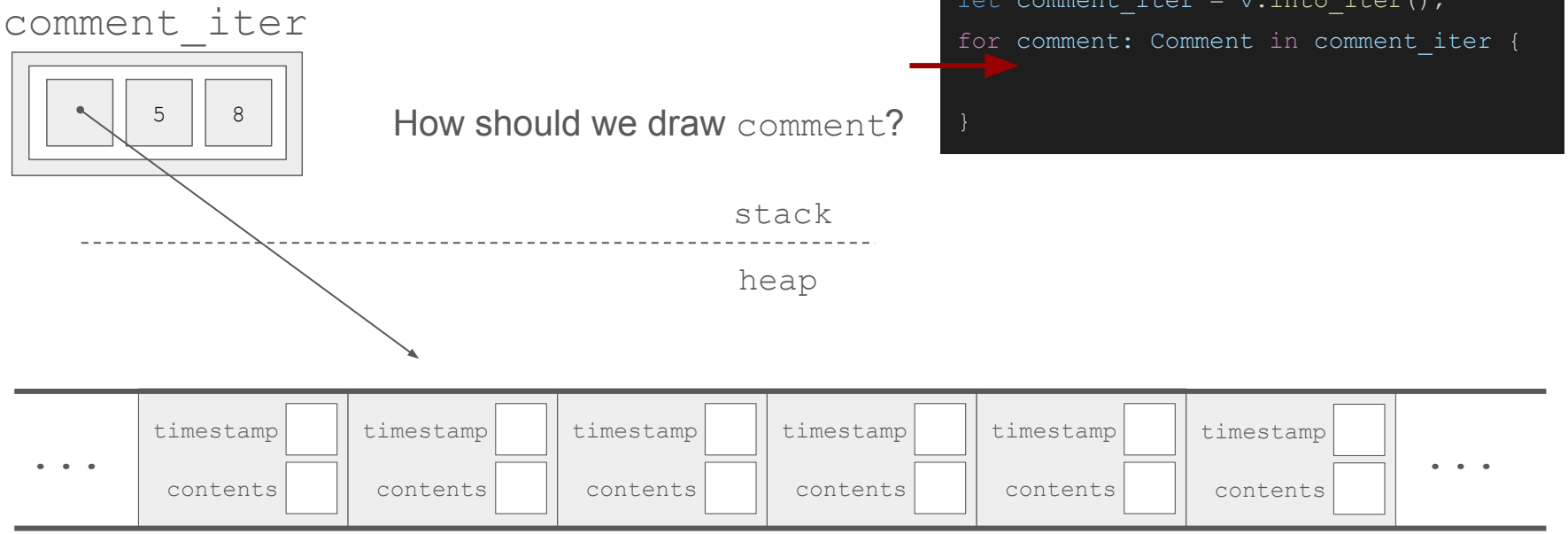


Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector

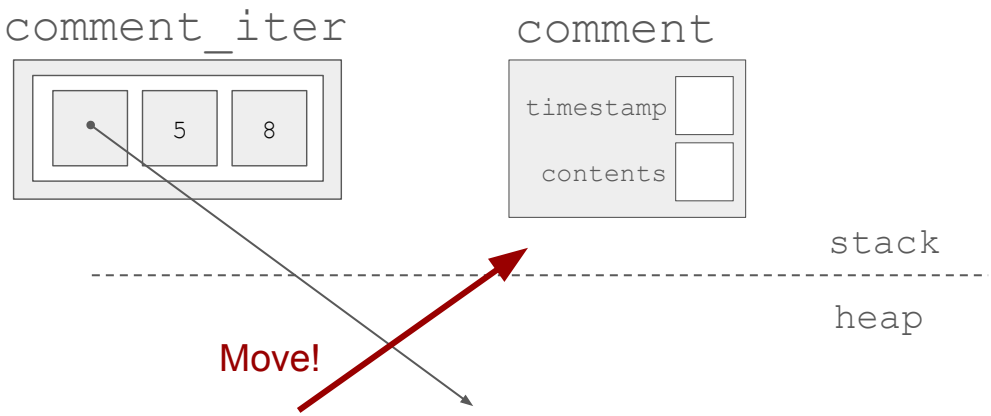
```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```

How should we draw `comment`?

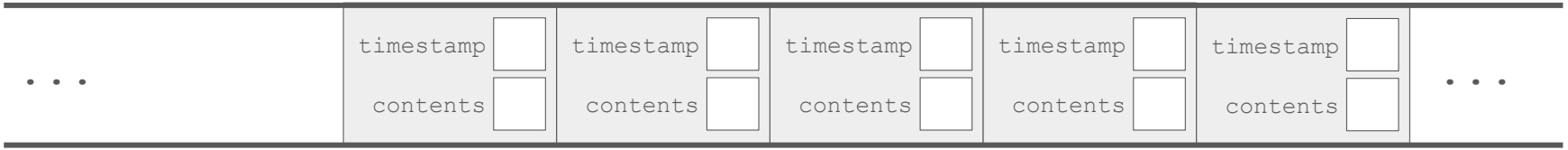


Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector

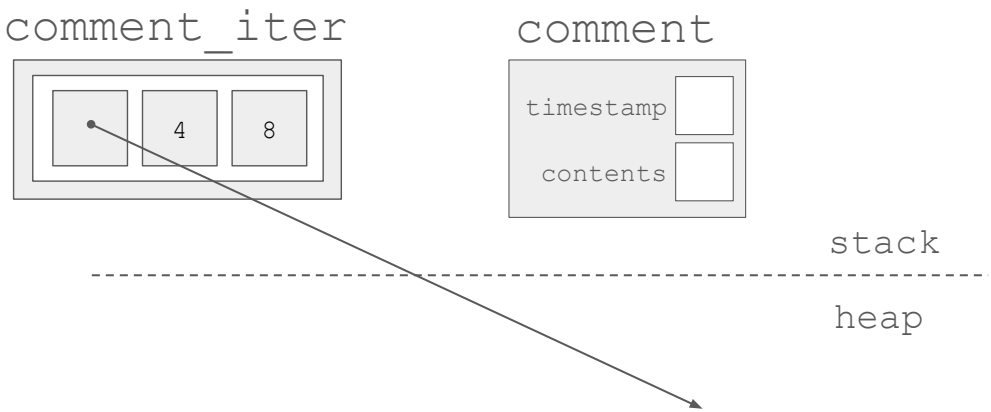



```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```

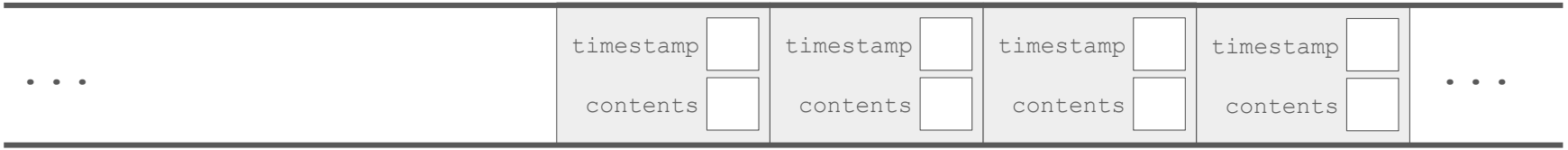


Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector

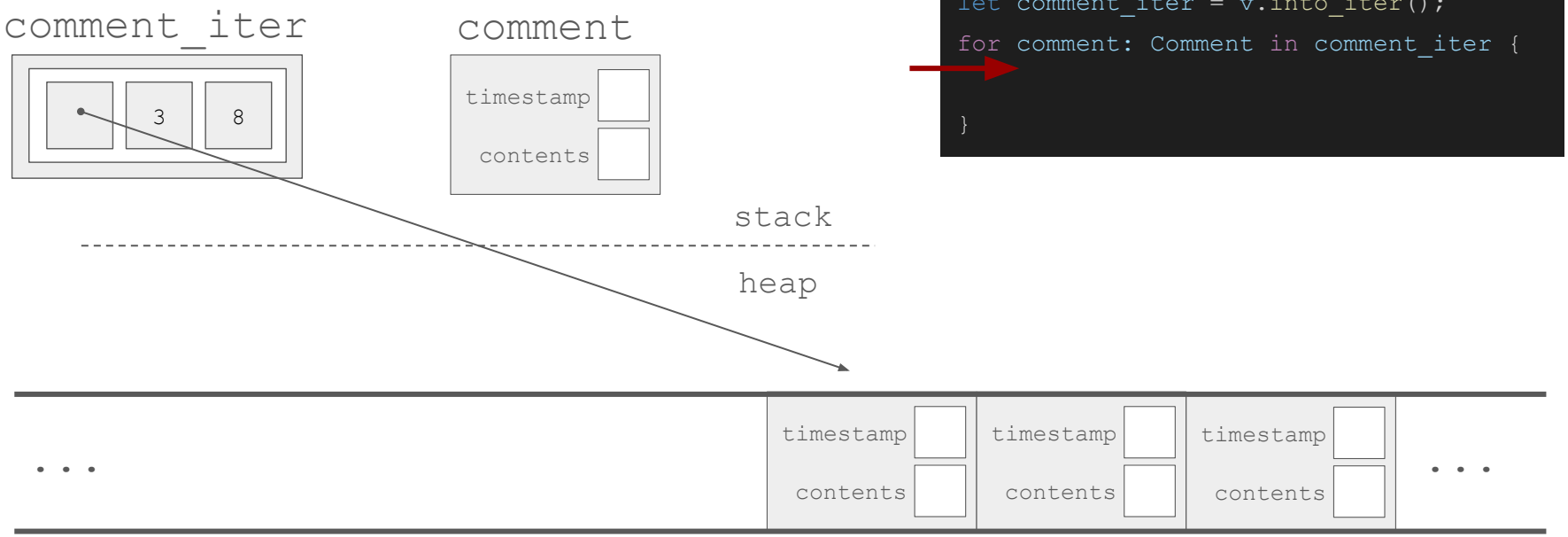



```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```



Ownership and Consumption

Let's think about how we can use an *owned* iterator over this vector



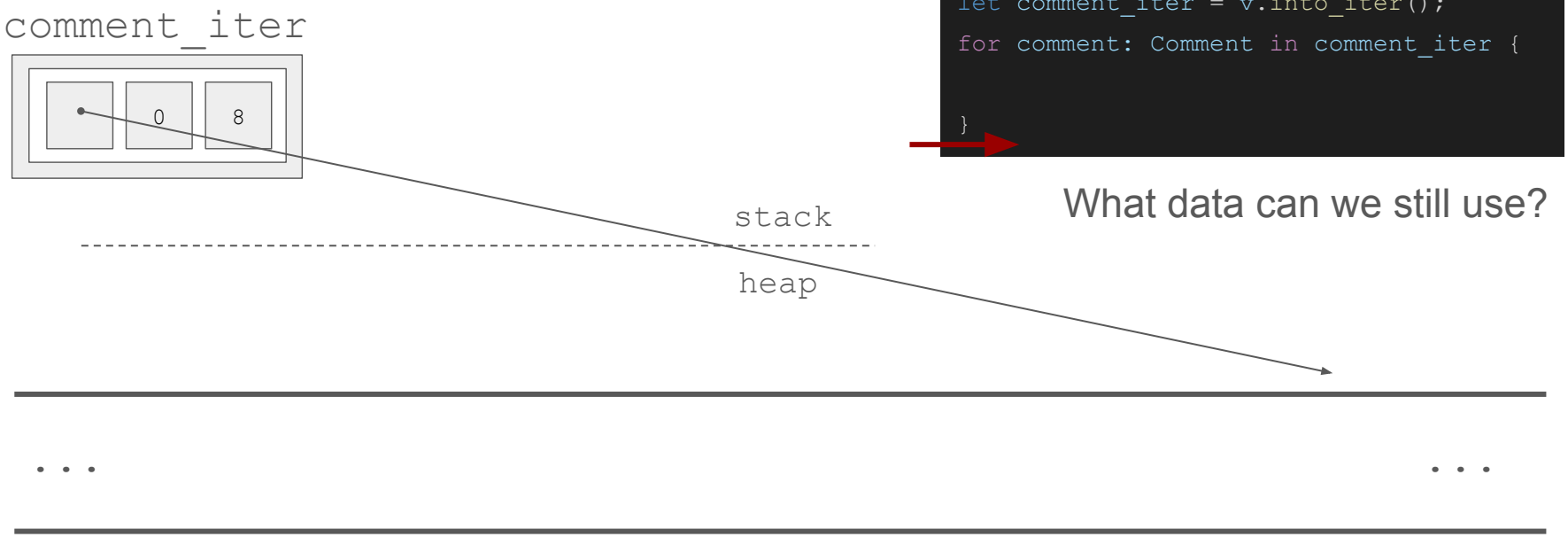

```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```

Ownership and Consumption

Let’s think about how we can use an *owned* iterator over this vector

```
let comment_iter = v.into_iter();
for comment: Comment in comment_iter {

}
```

What data can we still use?



Ownership and Consumption

Some takeaways:

- If an associated method accepts `self`, it *consumes* the struct/enum as ownership is moved into the method.
  - Some PL stuff: “Linear Types” → the type is “used” and cannot be used again
- If an associated method accepts `&mut self`, or `&self`, it might limit what you can do!
- You explicitly decide what to use when, you explicitly decide what to allow where.
  - As a library dev, choosing `self`, `&mut self`, or `&self` depends on what you want to allow.
    - It’s a contract!
  - As a user, choosing the correct function determines what you can do.